INITIAL COMMITS

```
@echo off
setlocal
echo ============================================================
echo     Building CoreInfo C++ System Monitor Engine
echo ============================================================

set CXX=g++
where %CXX% >nul 2>nul
if %ERRORLEVEL% neq 0 (
    echo [ERROR] MinGW g++ compiler not found in PATH.
    echo Searching C:\MinGW\bin\g++.exe...
    if exist "C:\MinGW\bin\g++.exe" (
        set "PATH=C:\MinGW\bin;%PATH%"
    ) else (
        echo [ERROR] Could not find g++.exe. Please install MinGW or add
it to PATH.
        exit /b 1
    )
)

echo Compiling C++ source files with MinGW GCC...
g++ -std=c++14 -O2 ^
    src\main.cpp ^
    src\monitor.cpp ^
    src\priority_controller.cpp ^
    src\scheduler.cpp ^
    src\db_manager.cpp ^
    src\http_server.cpp ^
    -o core_monitor.exe ^
    -lpsapi -lws2_32

if %ERRORLEVEL% equ 0 (
    echo.
    echo [SUCCESS] Build completed successfully: core_monitor.exe
    exit /b 0
) else (
    echo.
```

```
    echo [ERROR] Compilation failed. Review compiler output above.
    exit /b 1
)
```

CarInfo - In-Vehicle Infotainment (IVI) Cockpit & Scheduling Suite

A high-performance automotive embedded systems monitoring, process priority management, and CPU scheduling simulation engine built in **C++14**, featuring an interactive **Automotive Cockpit Web Dashboard**, **Google Maps Navigation**, **Spotify In-Car Media Player**, **Rear-View Safety Camera**, and a **PostgreSQL** database bridge.

🚗 Automotive Architecture & Subsystems

This project models and benchmarks an **In-Vehicle Infotainment (IVI) Head Unit** (e.g. Android Automotive OS, Automotive Grade Linux / AGL, or QNX) running on automotive multicore SoCs (like Qualcomm Snapdragon Automotive or NXP i.MX8).

Automotive Subsystem	Process / Service	Priority Class	Memory (RAM)	Scheduling Behavior
Safety Camera	`rear_camera_feed.bin`	`REALTIME`	88.0 MB	**Highest Safety Priority.** Preempts all background apps when driver shifts to Reverse (`R`).
CAN-Bus Gateway	`can_bus_gateway.bin`	`HIGH`	42.1 MB	Vehicle sensor telemetry, speed arbitration, and battery monitoring.
Google Maps Navigation	`google_maps_nav.bin`	`ABOVE_NORMAL`	438.5 MB	GPS turn-by-turn routing & 3D vector tile rendering.
Spotify In-Car Media	`spotify_audio_dsp.bin`	`NORMAL`	194.2 MB	Ogg Vorbis audio decoder & Bluetooth streaming via **Round Robin**.
HVAC Climate Control	`hvac_climate_ui.bin`	`NORMAL`	96.4 MB	Dual-zone touchscreen temperature & blower controller.
Virtual Cockpit Sync	`instrument_cluster_sync.bin`	`HIGH`	64.8 MB	Head-Up Display (HUD) and digital speedometer telemetry.
Cloud Diagnostics	`ota_telematics_sync.bin`	`IDLE`	112.0 MB	Background 4G/5G over-the-air firmware & diagnostic sync.

🌟 Interactive Automotive Dashboard Features

1. **Virtual Cockpit Telemetry HUD**:
   - Live Vehicle Speedometer (\(64\text{ km/h}\)), High-Voltage Battery Range (\(86\%\)), Cabin Temperature (\(21.5^\circ\text{C}\)), and CAN-Bus Network Load (\(26.8\%\)).
   - **Interactive Gear Selector (`P`, `R`, `N`, `D`)**: Shifting to **`R` (Reverse)** immediately activates the Rear-View Camera with trajectory guidelines and elevates its OS priority class to `REALTIME`.
2. **Google Maps Navigation Widget**:
   - Visual dark-mode route map, current speed limit badge (\(80\text{ km/h}\)), upcoming turn navigation indicator, and interactive **"Recalculate Route"** button.
3. **Spotify In-Car Media Player**:
   - Automotive audio player with animated frequency waveform visualizers, track skipping, play/pause controls, and lossless audio status.
4. **Rear-View Camera / Safety Monitor**:
   - Dynamic trajectory overlay guidelines. Shifting gears demonstrates preemptive priority scheduling in real time.
5. **Hardware Performance Telemetry**:
   - Real-time 60-second rolling Canvas charts for IVI SoC Processor Utilization and RAM allocation.
6. **Automotive Process Explorer**:
   - Interactive table with category tags, memory usage indicators, and an interactive **"Set Priority"** modal to modify process priorities via Win32 `SetPriorityClass`.
7. **Automotive CPU Scheduling Studio**:
   - 1-Click test presets:
     - 🚗 **Scenario 1**: *Reverse Gear Safety Preemption* (Rear Camera Priority 1 preempts Spotify & Google Maps).
     - 🎵 **Scenario 2**: *Multi-App Round Robin* (Google Maps + Spotify + HVAC sharing processor slices fairly with quantum \(q=3\)).
     - ⚠️ **Scenario 3**: *CAN-Bus Emergency Sensor Interrupt*.
   - Dynamic **Gantt Chart** timeline, Average Turnaround Time (\(TAT = CT - AT\)), and Average Waiting Time (\(WT = TAT - BT\)).
8. **PostgreSQL Telemetry Synchronization**:
   - Automated export of vehicle speed, battery, IVI process states, and simulation traces into `vehicle_telemetry`, `system_metrics`, `process_snapshots`, and `scheduling_simulations`.

🚀 How to Run in VS Code

1. In VS Code, open the integrated terminal (<kbd>Ctrl</kbd> + <kbd>`</kbd>).
2. Run:
   ```powershell
   .\run.bat
   ```
3. Your browser will immediately open `http://localhost:8080` displaying the live CarInfo Infotainment Cockpit.

🛠️ Automated Testing & Rebuilding

- **Rebuild C++ Engine**:
  ```powershell
  .\build.bat
  ```
  (Or press <kbd>Ctrl</kbd> + <kbd>Shift</kbd> + <kbd>B</kbd> in VS Code)

- **Run Automated Vehicle API Test Suite**:
  ```powershell
  .\test_api.ps1
  ```

```
@echo off
setlocal
echo ============================================================
echo     Launching CoreInfo OS Monitor ^& Dashboard
echo ============================================================

if not exist "%~dp0core_monitor.exe" (
    echo [INFO] core_monitor.exe not found. Running build.bat first...
    call "%~dp0build.bat"
    if %ERRORLEVEL% neq 0 (
        echo [ERROR] Build failed. Cannot launch.
        pause
        exit /b 1
    )
)

echo [INFO] Opening CoreInfo Dashboard in your default web browser...
start http://localhost:8080

echo [INFO] Starting C++ Monitoring Core on port 8080...
echo [INFO] Press Ctrl+C in this terminal window to stop the engine.
"%~dp0core_monitor.exe" 8080
```

```
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "Run CoreInfo Engine (GDB/MinGW)",
            "type": "cppdbg",
            "request": "launch",
            "program": "${workspaceFolder}/core_monitor.exe",
            "args": ["8080"],
            "stopAtEntry": false,
            "cwd": "${workspaceFolder}",
            "environment": [],
            "externalConsole": true,
            "MIMode": "gdb",
            "miDebuggerPath": "C:/MinGW/bin/gdb.exe",
            "preLaunchTask": "Build CoreInfo (MinGW)",
            "setupCommands": [
                {
                    "description": "Enable pretty-printing for gdb",
                    "text": "-enable-pretty-printing",
                    "ignoreFailures": true
                }
            ]
        }
    ]
}
```

```
{
    "version": "2.0.0",
    "tasks": [
        {
            "label": "Build CoreInfo (MinGW)",
            "type": "shell",
            "command": "cmd",
            "args": [
                "/c",
                "${workspaceFolder}\\build.bat"
            ],
            "group": {
                "kind": "build",
                "isDefault": true
            },
            "problemMatcher": "$gcc",
            "detail": "Compile C++ Core Monitor engine using MinGW GCC"
        },
        {
            "label": "Run CoreInfo & Dashboard",
            "type": "shell",
            "command": "cmd",
            "args": [
                "/c",
                "${workspaceFolder}\\run.bat"
            ],
            "group": {
                "kind": "test",
                "isDefault": true
            },
            "presentation": {
                "reveal": "always",
                "panel": "new",
                "focus": true
            },
            "detail": "Run core_monitor.exe and open dashboard in browser"
        },
        {
            "label": "Run API Verification Tests",
            "type": "shell",
```

```
            "command": "powershell",
            "args": [
                "-ExecutionPolicy",
                "Bypass",
                "-File",
                "${workspaceFolder}\\test_api.ps1"
            ],
            "problemMatcher": [],
            "detail": "Run automated REST API test suite"
        }
    ]
}
```

SETTTING.JSON

```
{
    "files.associations": {
        "*.h": "cpp",
        "*.cpp": "cpp"
    },
    "terminal.integrated.defaultProfile.windows": "Command Prompt"
}
```

```
{
    "configurations": [
        {
            "name": "Win32 MinGW",
            "includePath": [
                "${workspaceFolder}/src/**",
                "C:/MinGW/include/**",
                "C:/MinGW/lib/gcc/mingw32/6.3.0/include/c++/**"
            ],
            "defines": [
                "_DEBUG",
                "UNICODE",
                "_UNICODE",
                "_WIN32_WINNT=0x0600",
                "WINVER=0x0600"
            ],
            "compilerPath": "C:/MinGW/bin/g++.exe",
            "cStandard": "c11",
            "cppStandard": "c++14",
            "intelliSenseMode": "windows-gcc-x86"
        }
    ],
    "version": 4
}
```